



UNIVERSIDADE VIRTUAL DO ESTADO DE SÃO PAULO

Curso de Ciências de Dados e Engenharia da Computação

Disciplina: Projeto Integrador I

SISTEMA DE CONTROLE OPERACIONAL CONTÁBIL

Documentação da API

Integrantes:

Caio Henrique Ferreira Cavalcanti

Emmanuele de Moura Peres

Lucas Magalhães Nunes

Renato da Silva Cunha

Vinicius Fernandes dos Santos

Wagner Silva Matutino

Tutora: Adriana Costa de Souza

Jacareí / Ubatuba – SP

2026

SISTEMA DE CONTROLE OPERACIONAL CONTÁBIL

Documentação da API

Documentação técnica apresentada na disciplina de Projeto Integrador I para o curso de Ciências de Dados e Engenharia da Computação da Universidade Virtual do Estado de São Paulo (UNIVESP).

Tutora: Adriana Costa de Souza

Sumário

1 GUIA PARA RODAR O DJANGO LOCALMENTE

1.1 Instalar ferramentas necessárias

1.1.1 Python

Baixar em: <https://www.python.org/downloads/>

- Durante a instalação, marcar a opção **Add Python to PATH**.
- Para verificar se a instalação foi bem-sucedida, escreva no terminal:

```
python --version
```

1.1.2 Git

Baixar em: <https://git-scm.com/downloads/>

- Para verificar se a instalação foi bem-sucedida, escreva no terminal:

```
git --version
```

1.2 Clonar o projeto do GitHub

No terminal, escreva:

```
git clone https://github.com/CaioSix/ProjetoIntegradorI.git
```

E entre na pasta:

```
cd ProjetoIntegradorI
```

1.3 Criar ambiente virtual

Para criar, escreva no terminal:

```
python -m venv venv
```

Windows

Para ativar no CMD (Windows):

```
venv\Scripts\activate
```

Para ativar no PowerShell (Windows):

```
venv\Scripts\Activate.ps1
```

- Erro comum no PowerShell: *execution of scripts is disabled*.

- Para corrigir, escreva: `Set-ExecutionPolicy RemoteSigned`
- E depois: `y`

Linux / Mac

Para ativar no terminal UNIX:

```
source venv/bin/activate
```

1.4 Instalar dependências

Escrever no terminal:

```
pip install -r requirements.txt
```

1.5 Aplicar migrações do banco

Escrever no terminal:

```
python manage.py migrate
```

1.6 Carregar dados iniciais (fixtures)

Escrever no terminal:

```
python manage.py loaddata fixtures/dados_iniciais.json
```

1.7 Criar superusuário

Escrever no terminal:

```
python manage.py createsuperuser
```

Preencher usuário, e-mail e senha.

1.8 Rodar o servidor

Escrever no terminal:

```
python manage.py runserver
```

Acessar no navegador: <http://127.0.0.1:8000/api/>

2 ACESSAR A DOCUMENTAÇÃO

<http://127.0.0.1:8000/api/docs/>

3 BASE URL

<http://127.0.0.1:8000/api/>

4 AUTENTICAÇÃO

4.1 Login

POST /api/login/

Body:

```
{
  "username": "seu_usuario",
  "password": "sua_senha"
}
```

Resposta:

```
{
  "expiry": "2026-04-27T15:00:00-03:00",
  "token": "abc123..."
}
```

Uso do token:

Todas as rotas protegidas exigem a autorização no header:

```
Authorization: Token abc123...
```

Status:

- 200 OK
- 400 Bad Request

4.2 Logout

POST /api/logout/

Status:

- 204 No Content
- 401 Unauthorized

5 TAREFAS

5.1 Listar tarefas

GET /api/tarefas/

Query params (filtros):

```
?status=PENDENTE
?empresa=1
?competencia=1
?obrigacao=1
```

Resposta:

```
{
  "count": 33,
  "next": "http://localhost:8000/api/tarefas/?page=2",
  "previous": null,
  "results": [
    {
      "id": 37,
      "empresa_id": 7,
      "empresa_nome": "Delta Industria",
      "obrigacao_nome": "FGTS",
      "obrigacao_id": 17,
      "status": "PENDENTE",
      "status_prazo": "19 dias de atraso",
      "prazo": "2026-04-07",
      "prazo_formatado": "07/04/2026",
      "dias_prazo": -19,
      "prioridade": 1
    }
  ]
}
```

Status:

- 200 OK
- 401 Unauthorized

5.2 Listar tarefas pendentes (priorizadas)

GET /api/tarefas/pendentes/

- Apenas tarefas com status PENDENTE;
- Ordenadas por prioridade + prazo;
- Paginado.

5.3 Detalhe da tarefa

GET /api/tarefas/{id}/

Resposta:

```
{
  "id": 50,
  "empresa_id": "10",
  "empresa_nome": "Anabelle S.A.",
```

```

    "obrigacao_nome": "GUIA PARCELAMENTO",
    "obrigacao_id": 21,
    "status": "CONCLUIDA",
    "status_prazo": "Sem prazo",
    "prazo": null,
    "prazo_formatado": null,
    "dias_prazo": null,
    "prioridade": 5
}

```

5.4 Concluir tarefa

PATCH /api/tarefas/{id}/concluir/

Resposta:

```

{
  "mensagem": "Tarefa concluida com sucesso",
  "data": {
    "id": 37,
    "status": "CONCLUIDA",
    "concluida_em": "2026-04-27"
  }
}

```

Regras:

- Não pode concluir se já estiver concluída;
- Não pode concluir se estiver dispensada.

Status:

- 200 OK
- 400 Bad Request
- 401 Unauthorized

5.5 Reabrir tarefa

PATCH /api/tarefas/{id}/reabrir/

Regras:

- Não pode reabrir se estiver pendente;
- Não pode reabrir se estiver dispensada.

5.6 Dispensar tarefa

PATCH /api/tarefas/{id}/dispensar/

6 EMPRESAS

6.1 Listar empresas

GET /api/empresas/

6.2 Criar empresa

POST /api/empresas/

Body:

```
{
  "nome": "W-MAX",
  "codigo": "cod111",
  "tipo": "SN_COM_FOLHA"
}
```

7 DASHBOARD

7.1 Visão geral

GET /api/dashboard/

Resposta:

```
{
  "results": [
    {
      "empresa_id": 7,
      "empresa_nome": "Delta Industria",
      "tipo": "LUCRO_PRESUMIDO",
      "pendentes": 5,
      "concluidas": 0,
      "tarefas": [
        {
          "id": 37,
          "empresa_id": 7,
          "obligacao_id": 17,
          "obligacao_nome": "FGTS",
          "status": "DISPENSADA",
          "prazo": "2026-04-07"
        }
      ]
    }
  ]
}
```

Status de tarefa:

- PENDENTE
- CONCLUIDA
- DISPENSADA

Prioridade:

- 1 – atrasada
- 2 – vence hoje
- 3 – próximos 7 dias
- 4 – futuro
- 5 – sem prazo

8 EXEMPLO NO FRONT-END COM AXIOS

8.1 Login

```
const response = await axios.post('/api/login/', {
  username,
  password
})
const token = response.data.token
```

8.2 Requisição autenticada

```
axios.get('/api/tarefas/', {
  headers: {
    Authorization: 'Token ${token}'
  }
})
```

8.3 Concluir tarefa

```
axios.patch('/api/tarefas/${id}/concluir/', {}, {
  headers: {
    Authorization: 'Token ${token}'
  }
})
```